UNIVERSIDAD DE SAN BUENAVENTURA

Optativa II · Ingeniería de Software

MARKETPLACE

Documentación Firebase

Configuración, modelo de datos y reglas de seguridad

Basado en IEEE 1016 (vista de datos) · Código MKT-FIR-001 · Versión 2.0

Integrante	Rol Scrum
Sarah Isabella Peña	Product Owner
Juan Carlos Diaz	Scrum Master
Sebastian Grande Moreno	Developer

Docente: Ing. Jairo Armando Salcedo Aranda

Bogotá D.C., 3 de noviembre de 2026

Control de versiones

Versión	Fecha	Autor	Descripción
1.0	02/11/2026	Equipo Marketplace	Versión inicial: autenticación, catálogo, carrito, pedidos y chatbot.
2.0	03/11/2026	Equipo Marketplace	Perfiles Comprador y Administrador, favoritos, historial de pedidos, panel y gestión de catálogo.

Tabla de contenido

Control de versiones....2
Tabla de contenido....2
1. Resumen....3
2. Authentication....3
3. Modelo de datos en Firestore....4
3.1 users/{uid}....4
3.2 carts/{uid}....5
3.3 orders/{id}....5
4. Reglas de seguridad (sin cambios)....5
4.1 Compatibilidad de las nuevas funciones con las reglas....6
5. Verificación en la consola....6
6. Evolución sugerida (requiere autorización)....6

1. Resumen

Marketplace usa **Firebase Authentication** (correo y contraseña) y **Cloud Firestore**. En la versión 2.0 los perfiles se añadieron **sin modificar** la configuración del proyecto ni las reglas de seguridad: solo se agregaron campos nuevos a documentos que cada usuario ya podía escribir.

Dato	Proyecto
Project ID	marketplace-bdf94
Paquete Android	com.optativa.marketplace
Plan	Spark (gratuito)
Servicios	Authentication (Email/Password) · Cloud Firestore
Archivo de configuración	app/google-services.json
Dependencias	firebase-bom 33.1.2 · firebase-auth · firebase-firestore

2. Authentication

Operación	Método del SDK	Dónde
Registro	createUserWithEmailAndPassword + updateProfile(displayName)	register()
Login	signInWithEmailAndPassword	login()
Sesión persistente	FirebaseAuth.currentUser	Inicialización del ViewModel
Recuperar / cambiar contraseña	sendPasswordResetEmail	resetPassword(), sendPasswordChange()
Editar nombre	updateProfile(displayName)	updateProfile()
Cerrar sesión	signOut	logout()

Los errores del SDK se traducen a español con `AuthErrors.message(code)`.

3. Modelo de datos en Firestore

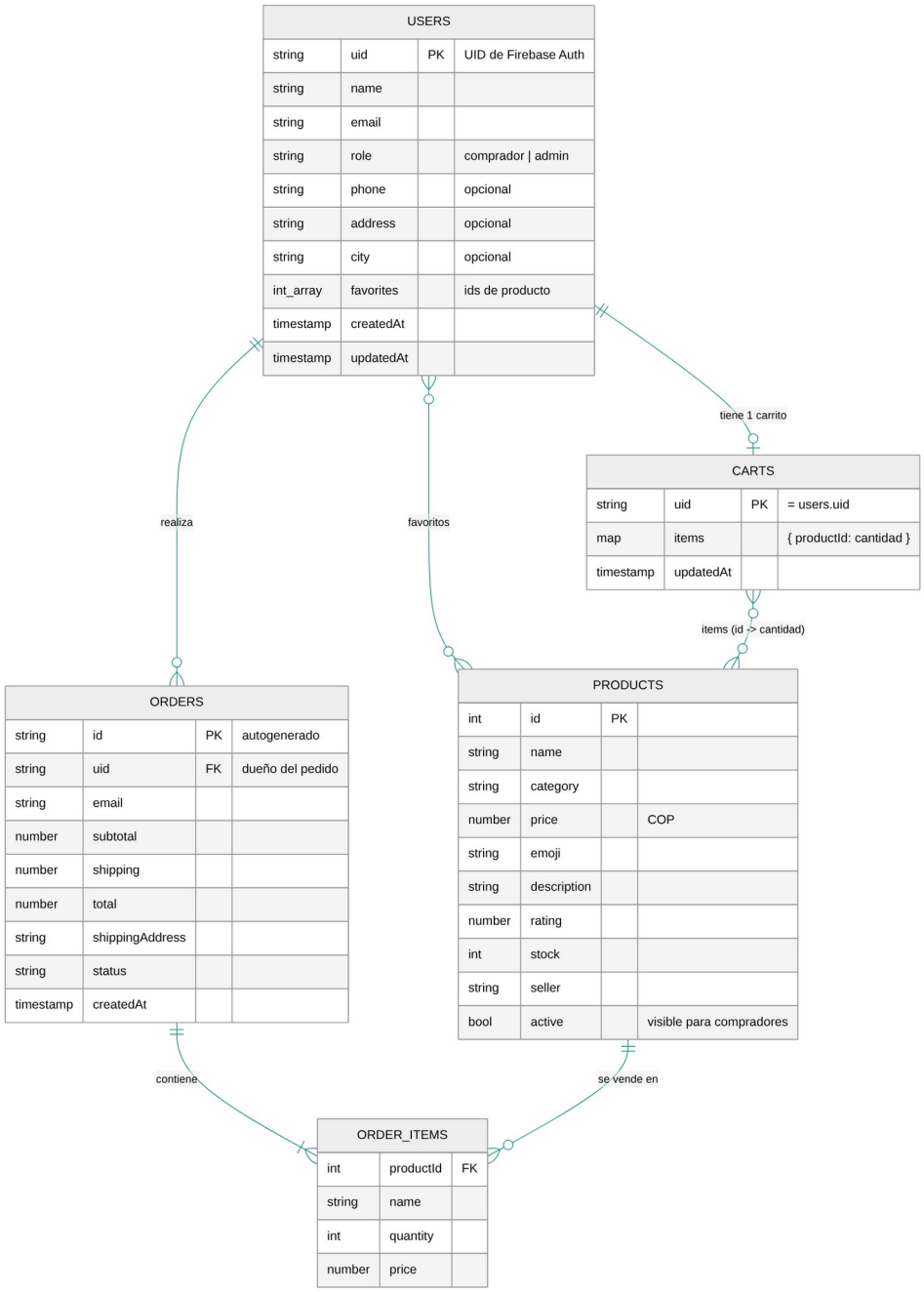


Figura 1. Colecciones y relaciones

3.1 users/{uid}

Campo	Tipo	Obligatorio	Descripción	Desde
name	string	Sí	Nombre visible	v1.0
email	string	Sí	Correo de la cuenta	v1.0
createdAt	timestamp	Sí	Fecha de registro (servidor)	v1.0

Campo	Tipo	Obligatorio	Descripción	Desde
role	string	Sí	"comprador" o "admin" (informativo)	v2.0
phone	string	No	Teléfono de contacto	v2.0
address	string	No	Dirección de envío	v2.0
city	string	No	Ciudad de envío	v2.0
favorites	array<number>	No	Ids de productos favoritos	v2.0
updatedAt	timestamp	No	Última edición del perfil	v2.0

3.2 carts/{uid}

Campo	Tipo	Descripción
items	map<string, number>	Id de producto → cantidad. Ej.: { "3": 2, "5": 1 }
updatedAt	timestamp	Última modificación

3.3 orders/{id}

Campo	Tipo	Descripción	Desde
uid	string	Dueño del pedido (validado por las reglas)	v1.0
email	string	Correo del comprador	v1.0
items	array<map>	{ productId, name, quantity, price }	v1.0
total	number	Total pagado	v1.0
createdAt	timestamp	Fecha del pedido	v1.0
subtotal	number	Suma de productos	v2.0
shipping	number	Costo de envío	v2.0
shippingAddress	string	Dirección y ciudad del perfil	v2.0
status	string	Estado («Confirmado»)	v2.0

4. Reglas de seguridad (sin cambios)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{uid} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
    match /carts/{uid} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
    match /orders/{orderId} {
      allow create: if request.auth != null && request.resource.data.uid ==
request.auth.uid;
      allow read: if request.auth != null && resource.data.uid == request.auth.uid;
      allow update, delete: if false;
```

```
    }
  }
}
```

4.1 Compatibilidad de las nuevas funciones con las reglas

Función v2.0	Operación	¿Permitida?	Motivo
Editar perfil	set(merge) en users/{uid} propio	Sí	El usuario escribe su propio documento.
Favoritos	set(merge) favorites en users/{uid}	Sí	Ídem.
Completar rol	set(merge) role en users/{uid}	Sí	Ídem; el rol no da permisos.
Historial	orders where uid == auth.uid	Sí	La consulta coincide con la regla de lectura.
Pedido con dirección	create orders con uid propio	Sí	Los campos extra no afectan la regla.
Catálogo admin	Sin Firestore (local)	N/A	Decisión DD-02.

> Por qué el rol no viene de Firestore: como las reglas permiten a cada usuario editar su `users/{uid}`, cualquiera podría escribir `role: "admin"`. Por eso la app decide el rol con `AdminConfig` y el campo `role` es solo informativo para la consola.

5. Verificación en la consola

Qué comprobar	Dónde
Usuarios registrados	Authentication → Users
Perfil, rol, favoritos	Firestore → Datos → users → (uid)
Carrito	Firestore → Datos → carts → (uid)
Pedidos con dirección y estado	Firestore → Datos → orders

6. Evolución sugerida (requiere autorización)

- Colección `products` con lectura pública y escritura solo para administradores.
- Rol por **Custom Claims** asignado desde Cloud Functions o Admin SDK.
- Regla de lectura de `orders` para administradores, para un panel de ventas global.
- Transacciones para descontar stock de forma atómica entre dispositivos.